D Y PATIL UNIVERSITY
NAVI MUMBAI

Subject Name : Machine Learning

Unit No: 2

Unit Name : Supervised Learning: Regression and Classification

Faculty Name : Dr. Ritu Jain

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Unit No.: 2.3

Faculty Name :

Classification

Classification

- Email: Spam / Not Spam?
- Online Transactions: Fraudulent (Yes / No)?
- Tumor: Malignant / Benign ?

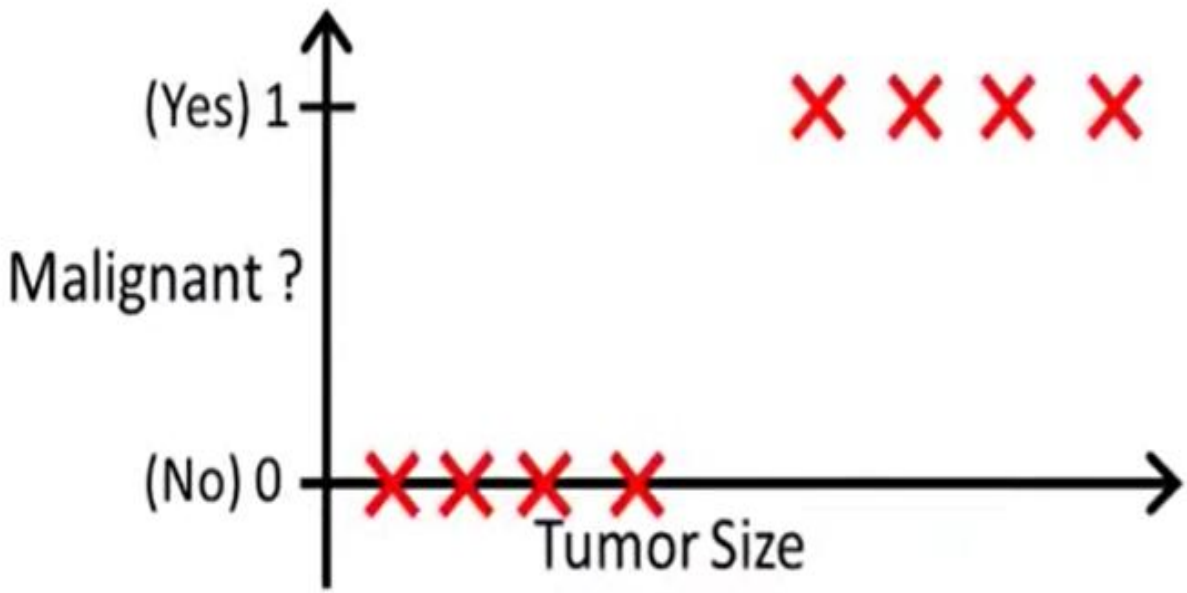
$y \in \{0, 1\}$

0: "Negative Class" (e.g., benign tumor)

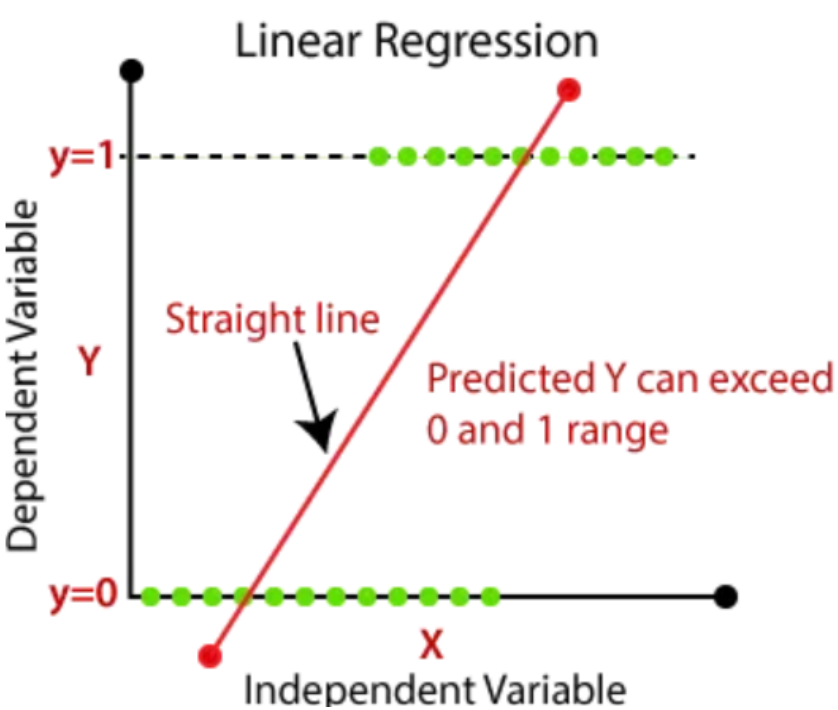
1: "Positive Class" (e.g., malignant tumor)

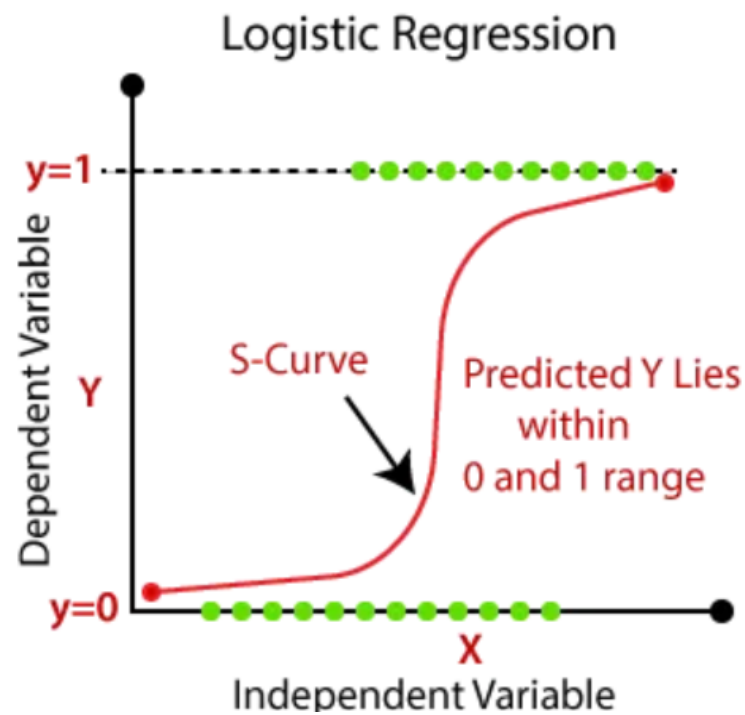
Classification: Logistic Regression

$$h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$



Classification: Logistic Regression





Logistic Regression: Classification Algorithm

- We could approach the classification problem ignoring the fact that y is discrete-valued, and use our old linear regression algorithm to try to predict y given x.
- However, it is easy to construct examples where this method performs very poorly (due to outliers).
- Intuitively, it also doesn't make sense for $h_\theta(x)$ to take values larger than 1 or smaller than 0 when we know that $y \in \{0, 1\}$.
- To fix this, let's change the form for our hypotheses $h_\theta(x)$ to satisfy $0 \leq h_\theta(x) \leq 1$. This is accomplished by plugging $\theta^T x$ into the Logistic Function (also known as Sigmoid Function).

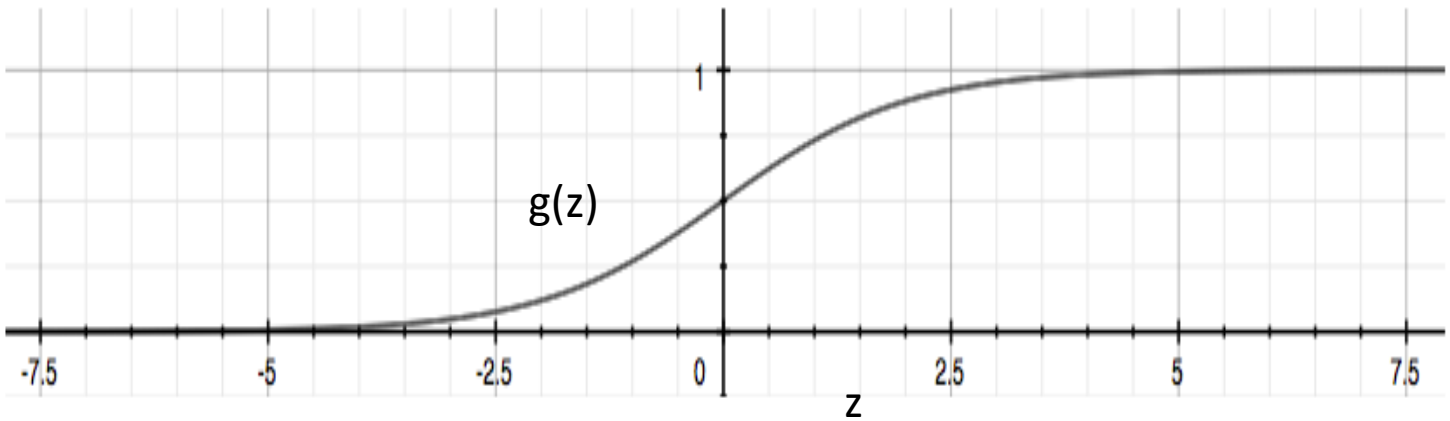
Logistic Regression: Classification Algorithm

- Logistic regression can only be applied on linearly separable data.
- We use equation of linear regression, $h_\theta(x) = (\theta^T x)$
- For classification hypothesis representation, $\mathbf{h_\theta(x) = g((\theta^T x))}$ ---- eq1
  - Where we define $g(z) = 1/(1 + e^{-z})$, where z is a real number --- eq2
    - This is the **sigmoid function**, or the **logistic function**
- Combining eq1 and eq2, we get,

$$
\begin{aligned}
h_\theta(x) &= g(\theta^T x) \\
&= \frac{1}{1 + e^{-\theta^T x}}
\end{aligned}
$$

Sigmoid Function

The following image shows us what the sigmoid function looks like:



The function g(z), shown here, maps any real number to the (0, 1) interval, making it useful for transforming an arbitrary-valued function into a function better suited for classification.

Logistic Regression

- $h_\theta(x)$ will give us the **probability** that our output is 1.

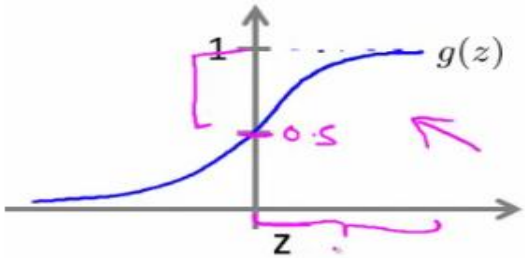
$h_\theta(x)$ = estimated probability that y = 1 on input x

- For example, $h_\theta(x) = 0.7$ gives us a probability of 70% that our output is 1.
- Our probability that our prediction is 0 is just the complement of our probability that it is 1 (e.g. if probability that it is 1 is 70%, then the probability that it is 0 is 30%).

$$h_\theta(x) = P(y = 1|x; \theta) = 1 - P(y = 0|x; \theta)$$
$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$

Decision Boundary

- The **decision boundary** is the line that separates the area where y = 0 and where y = 1. It is created by our hypothesis function.
- Gives a better sense of what the hypothesis function is computing.
- One way of using the sigmoid function is:
  - When the probability of y is greater than 0.5 then we can predict y = 1
  - Else we predict y = 0
- **When is it exactly that $h_\theta(x)$ is greater than equal to 0.5?**
  - Look at sigmoid function
    - g(z) is greater than or equal to 0.5, whenever z is greater than or equal to 0
    - $z = (\theta^T x)$
    - So when $\theta^T x >= 0$, Then $h_\theta(x) >= 0.5$; i.e y = 1



- So what we've shown is that the **hypothesis predicts y = 1 when $\theta^T x >= 0$.**
- The corollary of that when **$\theta^T x <0$ then the hypothesis predicts y = 0**

Decision Boundary

- The way our logistic function g behaves is that when its input is greater than or equal to zero, its output is greater than or equal to 0.5

$$g(z) \geq 0.5$$
$$when\ z \geq 0$$

- Remember,

$$z = 0, e^0 = 1 \Rightarrow g(z) = 1/2$$
$$z \to \infty, e^{-\infty} \to 0 \Rightarrow g(z) = 1$$
$$z \to -\infty, e^{\infty} \to \infty \Rightarrow g(z) = 0$$

$g(z) = 1/(1 + e^{-z})$

- So if our input to g is $\theta^T x$, then that means:

$$h_\theta(x) = g(\theta^T x) \geq 0.5$$
$$when\ \theta^T x \geq 0$$

- From these statements we can now say:

$$\theta^T x \geq 0 \Rightarrow y = 1$$
$$\theta^T x < 0 \Rightarrow y = 0$$

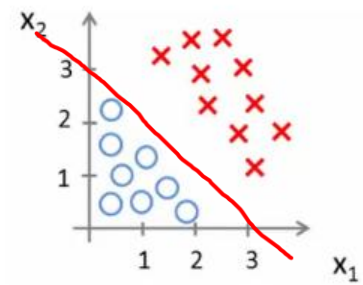
The decision boundary is a property of the hypothesis

Means we can create the boundary with the hypothesis and parameters without any data

Later, we use the data to determine the parameter values

Decision Boundary

- Suppose $h_\theta(x) = g(\theta_o + \theta_1 x_1 + \theta_2 x_2)$
- So, for example
  - $\theta_0 = -3$
  - $\theta_1 = 1$
  - $\theta_2 = 1$
- So our parameter vector is a column vector with the above va
  - So, θ^T is a row vector = [-3,1,1]
- What does this mean?
  - The z here becomes $\theta^T x$
  - We predict "y = 1" if
    - $-3x_0 + 1x_1 + 1x_2 >= 0$
    - $-3 + x_1 + x_2 >= 0$
- We can also re-write this as
  - If $(x_1 + x_2 >= 3)$ then we predict y = 1
  - If we plot
    - $x_1 + x_2 = 3$ we graphically plot our **decision boundary. It** is the set of points where $h_\theta(x) = 0.5$ exactly



Logistic Regression: Cost Function

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $\quad x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix} \quad x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

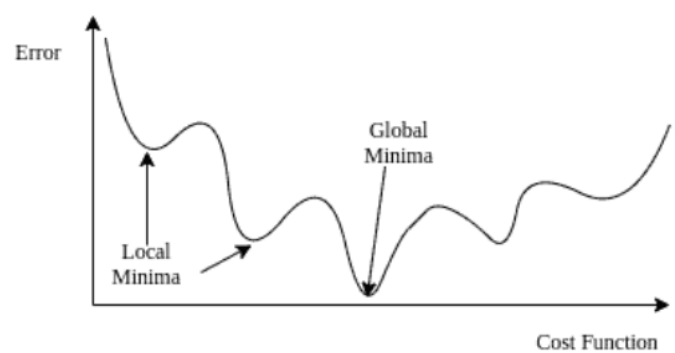
How to choose parameters θ ?

Logistic Regression: Cost Function

- Linear regression uses the following function to determine θ: $$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\frac{1}{2}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$
- Instead of writing the squared error term, we can write
  - If we define "cost()" as;
    - $\text{cost}(h_\theta(x^i), y) = 1/2(h_\theta(x^i) - y^i)^2$
    - Which evaluates to the cost for an individual example using the same measure as used in linear regression
  - We can **redefine J(θ) as** $$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$
    - Which, appropriately, is the sum of all the individual costs over the training data (i.e. the same as linear regression)
- To further simplify it we can get rid of the superscripts
  - So $$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\text{Cost}(h_\theta(x), y)$$

Logistic Regression: Cost Function

- We use the cost function to measure how close the model's predictions are to the actual outputs.
- In linear regression, we use mean squared error (MSE) as the cost function.
- But in logistic regression, using the mean of the squared differences between actual and predicted outcomes as the cost function might give a wavy, non-convex solution; containing many local optima:



Logistic Regression: Cost Function

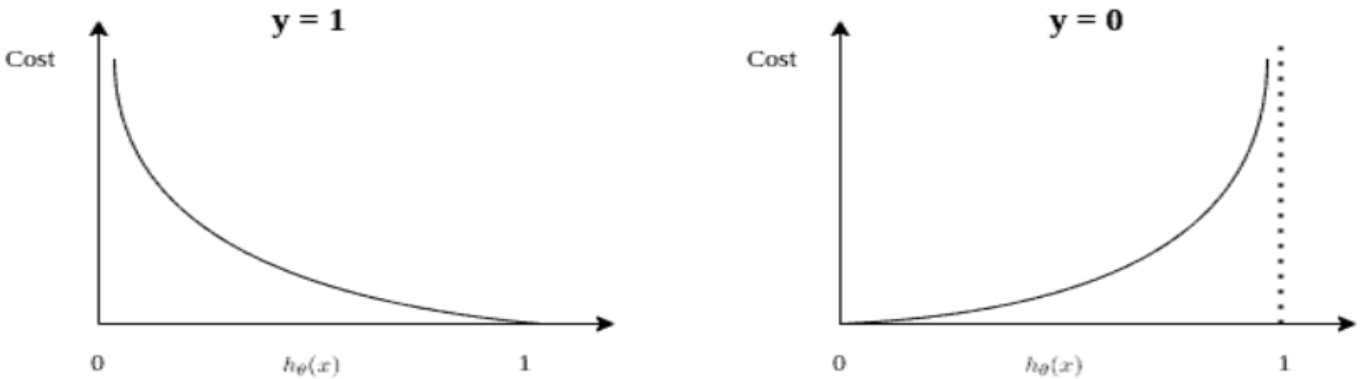
- In this case, finding an optimal solution with the gradient descent method is not possible.
- Instead, we use a logarithmic function to represent the cost of logistic regression.
- It is guaranteed to be convex for all input values, containing only one minimum, allowing us to run the gradient descent algorithm.
- When dealing with a binary classification problem, the logarithmic cost of error depends on the value of y.

Logistic Regression: Cost Function

- We can define the cost for two cases separately:

$$cost(h_\theta(x), y) = \begin{cases} -log(h_\theta(x)) & \text{, if } y = 1 \\ -log(1 - h_\theta(x)) & \text{, if } y = 0 \end{cases}$$

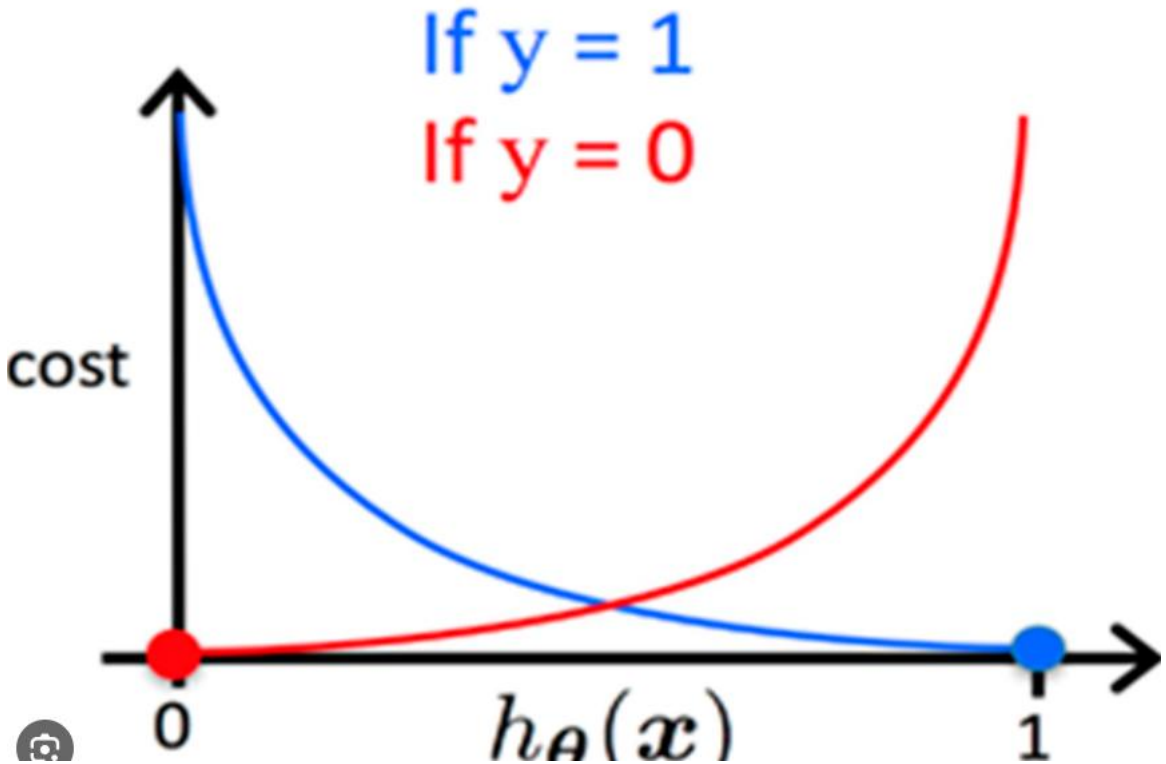
Which then results in:



Because when the actual outcome y = 1, the cost is 0 for h(x) = 1 and takes the maximum value for h(x) = 0.
Similarly, if y = 0, the cost is 0 for h(x) = 0.
As the output can either be {0} or {1}, we can simplify the equation to be:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(h_\theta(x^{(i)})) + (1 - y^{(i)})\log(1 - h_\theta(x^{(i)}))]$$

Logistic Regression: Cost Function



Logistic Regression: Parameters θ

- Now, we have to find parameters θ, such that, they minimize J(θ).
- We can use these parameters (θs) in our model for future predictions
- Then, if we're given some new example with set of features x, we can take the θ which we generated, and output our prediction using:

$$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$$

- This result will be:
  - p(y=1 | x ; θ)
    - Probability y = 1, given x, parameterized by θ

Gradient Descent: Minimizing Logistic Regression Cost Function

- **We will minimize the cost function using** gradient descent:
  - Repeatedly update each parameter using a learning rate

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

(simultaneously update all θ_j)

}

- After solving $\frac{\partial}{\partial \theta_j} J(\theta)$, we get:

Repeat {

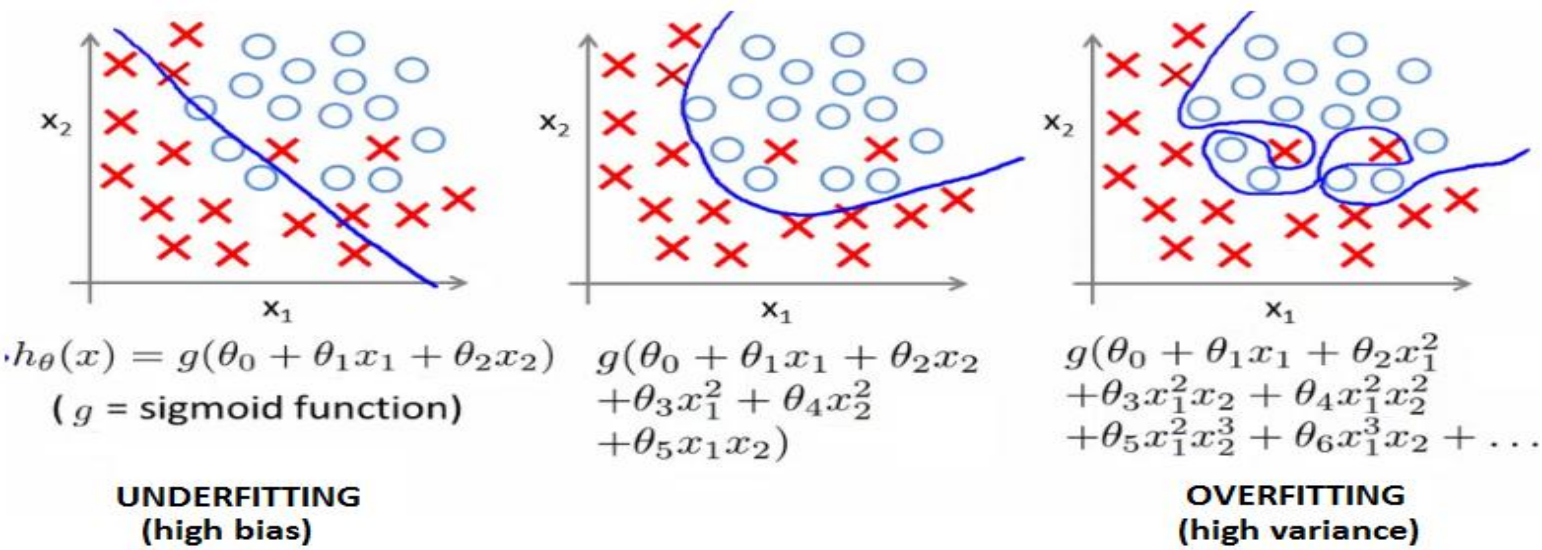
$$\theta_j := \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

}

The Problem of Overfitting

- **Overfitting with logistic regression**
- Same thing can happen to logistic regression
  - Sigmoidal function is an underfit.
  - But a high order polynomial gives and overfitting (high variance hypothesis)



L2 Regularization for logistic regression

- We saw earlier that logistic regression can be prone to overfitting with lots of features
- Logistic regression cost function is as follows:

$$J(\theta) = -\left[\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)} + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))\right]$$

To modify it we have to add an extra term. Regularized cost function:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))] + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$$

This has the effect of penalizing the parameters θ_1, θ_2 up to θ_n Means, like with linear regression, we can get what appears to be a better fitting lower order hypothesis

Performance Metrics for Classification

- **Evaluation metrics help us to measure the effectiveness of our models.**
- Classification Metrics are used for evaluating the performance of a classification model.
- **Key Classification Evaluation Metrics**

1. Confusion Matrix
2. Accuracy
3. Precision
4. Recall (Sensitivity)
5. F1 Score
6. ROC Curve & AUC

Performance Metrics for Classification: Confusion Metrics

- For a given sample observation, the *actual class* is either positive or negative.
- Similarly, the *predicted class* is also either positive or negative.
- We can visualize the outcome of a binary classification model using a confusion matrix, like the one presented above.
- A 2x2 matrix (for binary classification) that serves as the foundation for evaluation metrics.

Example of Confusion matrix

- A machine learning model is trained to predict tumor in patients. The test dataset consists of 100 people.

		ACTUAL	
		Negative	Positive
PREDICTION	Negative	60	8
	Positive	22	10

Confusion Matrix for tumor detection

Example of Confusion matrix

- **True Positive (TP)** – model correctly predicts the positive class (prediction and actual both are positive). In the above example, **10 people** who have tumors are predicted positively by the model.
- **True Negative (TN)** – model correctly predicts the negative class (prediction and actual both are negative). In the above example, **60 people** who don't have tumors are predicted negatively by the model.

		ACTUAL	
		Negative	Positive
PREDICTION	Negative	60	8
	Positive	22	10

Confusion Matrix for tumor detection

Example of Confusion matrix

- **False Positive (FP)** – model gives the wrong prediction of the negative class (predicted-positive, actual-negative). In the above example, **22 people** are predicted as positive of having a tumor, although they don't have a tumor. FP is also called a **TYPE I** error.
- **False Negative (FN)** – model wrongly predicts the positive class (predicted-negative, actual-positive). In the above example, **8 people** who have tumors are predicted as negative. FN is also called a **TYPE II** error.

		ACTUAL	
		Negative	Positive
PREDICTION	Negative	60	8
	Positive	22	10

Confusion Matrix for tumor detection

Performance Metrics for Classification: Confusion matrix

- There are four categories associated with the actual and predicted class of an observation:
  - **True Positive (TP):** Both the actual and predicted values of the given observation are positive.
  - **False Positive (FP):** The given observation is negative but the predicted value is positive.
  - **True Negative (TN):** Both the actual and predicted values of the given observation are negative.
  - **False Negative (FN)**: The given observation is predicted to be negative, despite, in fact, being positive.

		ACTUAL	
		Negative	Positive
PREDICTION	Negative	TRUE NEGATIVE	FALSE NEGATIVE
	Positive	FALSE POSITIVE	TRUE POSITIVE

Performance Metrics for Classification: Confusion matrix

- The diagonal of the confusion matrix presents the correct predictions. Obviously, we want the majority of our predictions to reside here.
- FPs and FNs are *classification errors*.
- **In statistics, FPs are referred to as Type I errors, and**
- **FNs are referred to as Type II errors.**
- In some cases, Type II errors are dangerous and not tolerable.

		ACTUAL	
		Negative	Positive
PREDICTION	Negative	TRUE NEGATIVE	FALSE NEGATIVE
	Positive	FALSE POSITIVE	TRUE POSITIVE

Confusion Matrix

Performance Metrics for Classification: Confusion Matrix

- With the help of these four values, we can calculate True Positive Rate (TPR), False Negative Rate (FPR), True Negative Rate (TNR), and False Negative Rate (FNR).

$$TPR = \frac{TP}{Actual\ Positive} = \frac{TP}{TP + FN}$$

$$FNR = \frac{FN}{Actual\ Positive} = \frac{FN}{TP + FN}$$

$$TNR = \frac{TN}{Actual\ Negative} = \frac{TN}{TN + FP}$$

$$FPR = \frac{FP}{Actual\ Negative} = \frac{FP}{TN + FP}$$

Performance Metrics for Classification: Confusion Matrix

- Even if data is imbalanced, we can figure out that our model is working well or not. For that, **the values of TPR and TNR should be high, and FPR and FNR should be as low as possible.**
- With the help of TP, TN, FN, and FP, other performance metrics can be calculated.

Performance Metrics for Classification: **Accuracy**

1. **Accuracy:**

- It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

$$\text{Accuracy} = \frac{TP + TN}{\#\ \text{samples}} = \frac{TP + TN}{TP + TN + FP + FN}$$

- While accuracy provides a quick snapshot, it can be misleading in cases of imbalanced datasets. For example, in a dataset with 90% class A and 10% class B, a model predicting only class A will still achieve 90% accuracy but it will fail to identify any class B instances.
- Accuracy is good but it gives a False Positive sense of achieving high accuracy.
- The problem arises due to the possibility of misclassification of minor class samples being very high.

Precision

- **Precision is the fraction of instances marked as positive that are actually positive. In other words, precision measures "how useful are the results of our classifier".** The mathematical notation is:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Otherwise speaking, a precision of 90% would mean when our classifier marks an e-mail as spam, it is indeed spam 90 out of 100 times.

Recall

- **Recall is the fraction of true positive instances that are marked to be positive. It measures "how complete the results are" — that is, which percentage of true positives are predicted as positive.** The representation is:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- That is to say, a recall of 90% would mean that the classifier correctly labels 90% of all the spam e-mails, hence 10% are marked as not spam.
- **Perfect precision is equivalent to no FPs (no Type I errors), while on the other hand, perfect recall means there are no FNs (no Type II errors).**

Precision and Recall

		Real Label	
		Positive	Negative
Predicted Label	Positive	True Positive (TP)	False Positive (FP)
	Negative	False Negative (FN)	True Negative (TN)

$$\text{Precision} = \frac{\sum \text{TP}}{\sum \text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\sum \text{TP}}{\sum \text{TP} + \text{FN}}$$

$$\text{Accuracy} = \frac{\sum \text{TP} + \text{TN}}{\sum \text{TP} + \text{FP} + \text{FN} + \text{TN}}$$

Calculation of Precision, Recall and Accuracy in the confusion matrix.

F1 Score

- Usually, having a balanced recall and precision is more important than having one type of error really low. That's why generally we take both precision and recall into consideration. Several classification measures are developed to identify this problem.
- For example, the **F-1 score is the harmonic mean of precision and recall.** It gives equal importance to Type I and Type II errors. The calculation is:

$$\text{F-1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

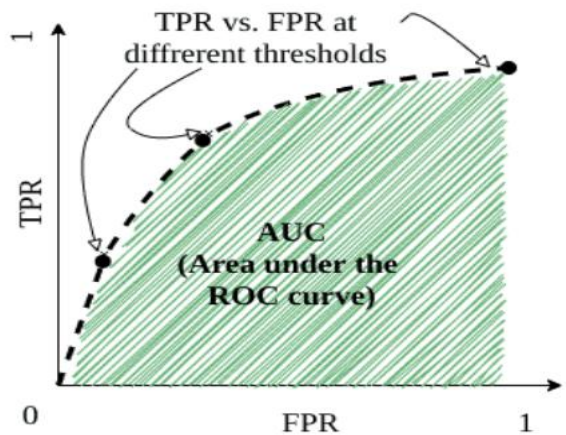
- When the dataset labels are evenly distributed, accuracy gives meaningful results. But if the dataset is imbalanced, like the spam e-mail example, we should prefer the F-1 score.
- Another point to keep in mind is; accuracy gives more importance to TPs and TNs. On the other hand, the F-1 score considers FN and FPs.

ROC Curve and AUC

- ROC curve (receiver operating characteristic curve). is a method to visualize the classification performance.
- The plot shows the classifier's success for different threshold values.
- In order to plot the ROC curve, we need to calculate the True Positive Rate (TPR) and the False Positive Rate (FPR), where:

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$



- The ROC curve represents the performance of a binary classifier by plotting the **True Positive Rate (TPR/Sensitivity)** on the Y-axis against the **False Positive Rate (FPR)** on the X-axis across all possible decision thresholds.
- The goal is typically to choose a threshold that maximizes the True Positive Rate (top-left) while minimizing the False Positive Rate (0 on X-axis).

Key: tp = True Positive, **tn** = True Negative, **fp** = False Positive, **fn** = False Negative

Metric Name	Metric Forumla	Code	When to use
Accuracy	$\textbf{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$	`tf.keras.metrics.Accuracy()` or `sklearn.metrics.accuracy_score()`	Default metric for classification problems. Not the best for imbalanced classes.
Precision	$\textbf{Precision} = \frac{tp}{tp + fp}$	`tf.keras.metrics.Precision()` or `sklearn.metrics.precision_score()`	Higher precision leads to less false positives.
Recall	$\textbf{Recall} = \frac{tp}{tp + fn}$	`tf.keras.metrics.Recall()` or `sklearn.metrics.recall_score()`	Higher recall leads to less false negatives.
F1-score	$\textbf{F1-score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$	`sklearn.metrics.f1_score()`	Combination of precision and recall, usually a good overall metric for a classification model.
Confusion matrix	NA	Custom function or `sklearn.metrics.confusion_matrix()`	When comparing predictions to truth labels to see where model gets confused. Can be hard to use with large numbers of classes.

Thank You!!!